Assignment

An assignment for class **MET CS 566** at Boston University, instructed by **Ming Zhang**. Student Zhang, Jin Yu submitted on **November 21, 2022**.

Question 1

An investor wants to invest 10 million dollars in real estate. He can select properties from a list of 4 houses in 4 different cities in Massachusetts. He wants to purchase as much as possible, and he wants to maximize the expected profits that he can get when he is going to sell the houses in the next year.

Table 1: 4 houses with their house prices and their expected home value growth in the next year.

Houses	1. Newton	2. Waltham	3. Watertown	4. Weston
House Values in Millions	3	2	4	3
Expected profit in the next year %	2	3	3	2

Describe what are the sub-problems here and what is the size/count of the sub-problems

The sub-problems are finding the best combination for purchasing as must as possible houses and maximizing the profit next year from selling them with a specific amount of money. Similar to the rope-cutting problem, the length of the rope is limited as we have a limited amount of money (10 million dollars), and we want to find the highest value we can obtain with some costs like a 2-long-length rope or 2 million dollars.

Describe which inputs to the knapsack problem (value and size) correspond to the inputs in the home purchasing problem.

Take the table in the question below as an example. We have $V[i, w]$ where i is the y-axis and w is the x-axis. i is the houses, which means the value for $i = 1$ is that we have to pay 3 million to purchase the house in Newton, and we expect that we can get a 2% profit next year. (which is $\$3,000,000 \times 2\% = \$60,000$. see the table below)

Houses	**1. Newton**	**2. Waltham**	**3. Watertown**	**4. Weston**
House Values in Millions	3	2	4	3
Expected profit in the next year %	2	3	3	2
Expected profit in the next year $	60,000	60,000	120,000	60,000

w is the amount of money we pay for purchasing houses. For example, $w = 1$ means 1 million dollars.

Fill up the following dynamic programming table and check the maximum possible profits.

$V[i, w]$	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**
$i = 0$	0	0	0	0	0	0	0	0	0	0	0
$i = 1$	0	0	0	60,000	60,000	60,000	60,000	60,000	60,000	60,000	60,000
$i = 2$	0	0	60,000	60,000	60,000	120,000	120,000	120,000	120,000	120,000	120,000
$i = 3$	0	0	60,000	60,000	120,000	120,000	180000	180000	180000	240000	240000
$i = 4$	0	0	60,000	60,000	120,000	120,000	180000	180000	180000	240000	240000

What is the optimal set of houses to buy and invest the money in?

The optimal sets include $(w = 9, i = 3), (w = 10, i = 3), (w = 9, i = 4), (w = 10, i = 4)$. I implement the solution in Python:

```
MILLION = 1000000

def knapsack(budget_limit, list_houses):
    matrix = [[0 for _ in range(budget_limit + 1)]
        for _ in range(len(list_houses) + 1)]

    list_houses_copied = list_houses.copy()
    list_houses_copied.insert(0, [0, 0])

    for index_house in range(len(list_houses_copied)):
        for budget in range(budget_limit + 1):
            if budget == 0 or index_house == 0:
                continue

            house_cost = list_houses_copied[index_house][0]
            house_percentage = list_houses_copied[index_house][1]
            house_profit = int(house_cost * MILLION * house_percentage)

            remain = budget - house_cost
            current_profit = house_profit if remain >= 0 else 0
            last_profit = matrix[index_house - 1][max(remain, 0)]

            matrix[index_house][budget] = \
                max(
                    matrix[index_house - 1][budget],
                    current_profit + last_profit
                )

    for item in matrix:
        print(item)


knapsack(10, [[3, 0.02], [2, 0.03], [4, 0.03], [3, 0.02]])
```

Question 2

A city planner is asked to organize grocery shops in a new city. The city has a straight-line main street that goes throughout the city. The city planner is asked where the city should provide permits to build new grocery shops so that people of the new city can have the shortest distance to their grocery markets. The population density is not constant along both sides of the main street. A higher density is around multiple city centers or crossroads alongside the main street.

Your task is to develop an algorithm, given the positions of the city centers and the number of grocery shops compute the least possible sum of all distances between each city center and its nearest grocery shops.

What are the sub-problems in this case? What are the counts of sub-problems? Provide a brief description of your solution.

Our mission is to compute the least possible sum of all distances between each city center and its nearest grocery shops, which means we have to obtain every distance from every given city center to its nearest grocery shop. For example, the costs (distance) from 2 and 4 to a shop located at 3 are both 1. The sub-

problems here are finding the lowest cost for shops located between 2 city centers. More precisely, we place shops in the city and find the best/lowest location for each of them and sum the distance up.

Write up your algorithm in Pseudo-code or python implementation.

```
def knapsack(number_shops , list_city_centers):
    number_city_centers = len(list_city_centers)

    if number_city_centers == 0:
        raise Exception('There is no city centers.')

    if number_shops >= number_city_centers:
        raise Exception('The number of shops should be' \
             'smaller than the number of city centers.')

    list_city_centers_copied = list_city_centers.copy()
    list_city_centers_copied.sort()

    cost = [[0 for _ in range(number_city_centers + 1)]
        for _ in range(number_city_centers + 1)]

    '''
    number_city_centers: 4

    for 0,1,2,3
        index_city_center: 0
            for 0,1,2,3
                > index_line: 0
                    middle = (0 + 0) / 2 = 0
                    for 0
                > index_line: 1
                    ...
                > index_line: 2
                    middle = (0 + 2) / 2 = 1
                    for 0,1,2
                > index_line: 3
                    middle = (0 + 3) / 2 = 1
                    for 0,1,2,3
                        >> index_inner: 0
                            delta = list_city_centers[0] - list_city_centers[1] = 1
                            cost[0 + 1][3 + 1] += delta (set to positive as no nega
                        >> index_inner: 1
                            delta = list_city_centers[1] - list_city_centers[1] = 2
                            cost[0 + 1][3 + 1] += delta
                        >> index_inner: 2
                            delta = list_city_centers[2] - list_city_centers[1] = 3
                            cost[0 + 1][3 + 1] += delta
                        >> index_inner: 3
                            delta = list_city_centers[3] - list_city_centers[1] = 6
                            cost[0 + 1][3 + 1] += delta

                        const[0 + 1][3 + 1] :: -(-1) + 0 + 1 + 4 = 6
            for 1,2,3
            ...
        index_city_center: 1
            ...
    '''
```

```
    for index_city_center in range(number_city_centers):
        for index_line in range(index_city_center, number_city_centers):
            middle = int((index_city_center + index_line) / 2)
            for index_inner in range(index_city_center, index_line + 1):
                delta = list_city_centers[index_inner] - \
                    list_city_centers[middle]
                cost[index_city_center + 1][index_line + 1] += delta * (-1 if delta

    dp_table = [[float('inf') for _ in range(number_city_centers + 1)]
        for _ in range(number_shops + 1)]

    dp_table[0][0] = 0

    for index_shop in range(1, number_shops + 1):
        for index_city_center in range(1, number_city_centers + 1):
            for index_line in range(index_city_center):
                dp_table[index_shop][index_city_center] = \
                    min(dp_table[index_shop][index_city_center],
                        dp_table[index_shop - 1][index_line] + cost[index_city_cent

    for item in dp_table:
        print(item)

    return dp_table[-1][-1]


print(knapsack(5, [1, 2, 3, 6, 7, 9, 11, 21, 40, 50]))
```

What is the run time complexity of your algorithm?

We use 2 for-loops with the given number of shops (ns) and number of city centers (ncc) in the dynamic-programming/knapsack-problem part, which causes the time complexity to be $\theta(ns \times ncc)$. The time complexity for computing cost table for shops between 2 city centers is $\theta(ncc^3)$ since we have 3 for-loops and they eventually iterate to the number of city centers. Therefore, the time complexity for the algorithm is $\theta(ns \times ncc + ncc^3 = ncc^3)$.